# Correct Optimizations: Phase 1 Report
# Based on Minotaur: A SIMD-Oriented Synthesizing Superoptimizer [1]

Kilian Schulz
*Technical University Munich*

Ilja Gamza
*Technical University Munich*

Tomás Gaspar
*Technical University Munich*

Claudia Dünzinger
*Technical University Munich*

## 1 Introduction

This report aims to give a summary of Liu et al. [1], evaluate their results and propose further research ideas for Minotaur.

## 2 Paper Summary

### 2.1 Context

Minotaur is a superoptimizer that tries to identify missed optimization opportunities of the LLVM compiler by slicing the generated Intermediate Representation (IR) and trying to synthesize new faster code. Its synthesizer creates improved machine code by generating and comparing equivalent but cheaper instruction sequences. To guarantee correctness of the optimization, Alive2 was extended to be able to formally verify the generated code. Additionally, to reduce computation overhead, Minotaur also caches discovered transformations for reuse in future compilations.

The use case of Minotaur is to further optimize already heavily optimized IR from previous translation steps, which still often contain suboptimal code segments, particularly for Single Instruction Multiple Data (SIMD) operations. The authors showed that even LLVM's advanced automatic-vectorizer produced redundant or inefficient SIMD instructions sequences. For those missed opportunities, Minotaur can improve suboptimal sections by replacing them with improved ones during the compilation. Some of those optimizations could be generalized and added to LLVM, Hence, enabling better and formally verified machine code generation through the use of Minotaur.

Before Minotaur, several superoptimizers already existed that searched the program space – either in assembly or LLVM IR – for more efficient instruction sequences, rather than relying on the rewrite rules used by traditional compilers. Common examples include Souper [2] – which focuses on scalar integer optimizations also using LLVM IR – and STOKE [3], which relies on stochastic search for faster x86 code block generation. However, those tools only addressed part of the problem. Souper detects only improvements for integer operations, but not for floating-point or SIMD instructions. STOKE's improvements include complex optimizations, but the tool relies on randomized search to find those, making them harder to generalize into peephole optimizations. Therefore, the combined challenge of performing complex vector or floating point optimizations, formally verifying the transformation and integrating those capabilities into production compilers remained unsolved, until Minotaur.

Designing an efficient and high performing superoptimizer included several challenges. First, the search space for replacements of suboptimal code sequences is enormous, making an exhaustive search too computationally expensive. Second, the formal checks add further computational cost – especially for complex SIMD and floating point operations. Minotaur relies on Satifiablility Module Theories (SMT) solvers for equivalency checks between the original code and the proposed optimization, but these solvers often struggle with large complex inputs. At the same time, a sufficiently large context window is needed to provide enough information to be able to find possible transformations. Since Minotaur aims for low level, hardware aware optimizations, the authors used detailed information about CPU pipelines to accurately model the performance. These difficulties of realizing high performance without too much computational cost required a multi stage process.

### 2.2 Contributions

Because today's SMT solvers cannot efficiently reason about entire LLVM functions, Minotaur instead operates on single instruction level. In order to achieve this, it breaks the LLVM function in Static Single-Assignment (SSA) form down into cuts, which are small loop-free subsets of a given instruction's dependencies, to capture context. The size of the cut influences the optimization, as small cuts contain not enough context to determine a better code sequence, while large cuts increase the solvers computation time dramatically. Minotaur's cut extraction is noticeably more capable than previous

ones since it is able to handle memory, vector and floating-point operations. In comparison, Souper could only handle scalar integer instructions and loop-free control flow [2], both of which are also handled by Minotaur. Through an empirical study, the authors selected the cutting depth of four, as it provided a good balance between finding most optimizations while still running in an acceptable amount of time. For each cut, a set of potential faster alternatives is synthesized. These are then verified with Alive2, which the authors extended to support x86 SIMD intrinsics. Lastly, for each optimization its speedup is estimated using a more accurate model (LLVM Machine Code Analyzer (MCA)). If a faster version is found, it is applied. In addition, the original and transformed code is cached for reuse in future compilations.

To measure how Minotaur's optimizations influence the overall performance of larger software projects, the authors benchmarked Minotaur on the SPEC CPU 2017, libgmp and libYUV on Intel and AMD hardware with LLVM 18. They determined a speedup of 1 - 7 percent, where integer SIMD fragments made up 75 percent of the discovered rewrites. When the compilation is performed with an empty cache, Minotaur increased compile times significantly. With a filled cache, however, only an additional 3 percent of computation time was needed. The authors found 15 simple and general floating-point and vector optimizations, which they then reported to the LLVM project.

# 3 Artifact Evaluation Results

## 3.1 Project Issues and Benchmarking Setup

The paper did not specify which version of Minotaur was used for the benchmarks. Since we could not determine the version used by the authors, we chose to evaluate the most up-to-date version of Minotaur[1]. One important difference between the most recent version and the version referenced in the paper, is that it relied on LLVM 18 whereas the newest one uses LLVM 20.

The most up to date version of Minotaur, as of writing, does not work out of the box. For it to not immediately crash on startup, the order in which the libraries are linked in Minotaur had to be changed. This changes the order in which the offending dynamic libraries are loaded on startup and thereby fixing the issue of undefined symbols.

Minotaur uses Alive2's LLVM interpreter which is listed as highly experimental [2] and caused multiple crashes, some of which where not reliably reproducible and could therefore not be resolved. In order to mitigate this, problematic source files were filtered. More about this is discussed in section 3.3.

Minotaur's caching seems to not fully work in the last release, as compiling a source file multiple consecutive times still leads to cache misses. This might be caused by differing inputs from LLVM's side.

Everything was compiled with the `-O3` and `-march=native` flags as described in the paper. We benchmarked libgmp-6.2.1, as the paper did, and a fitting version of libYUV [3], which is able to build with LLVM 20. All runs were performed with the same parameters as the paper, i.e. a cutting depth bound of four, total synthesis time of five minutes and a one minute timeout for individual SMT queries. The benchmarks were built and run on an AMD Ryzen 9 5900X, whereas the authors used an AMD Ryzen 9 5950X. The former has 12 cores running at up to 4.8 GHz, the latter has 16 cores running at up to 4.9 GHz.

## 3.2 libgmp

According to the paper, the build of libgmp with an empty cache took 440 minutes. Our build took approximately 1020 minutes while using a partly dirty cache from previous attempts. It seems unlikely that this discrepancy in build times could be caused by the difference in hardware. It is more likely that this is due to the authors using the offline mode of Minotaur, which we intentionally did not use. As discussed earlier, the cache did not work as expected and using the offline mode would effectively have doubled the compile time. The reason for this is that after creating the cuts, Minotaur tries to synthesize a faster version of each of them. After that is done, another run of Minotaur is required to apply the new cuts, but because the caching is not working properly, it leads to another long compilation where new code is synthesized.

After building libgmp, the provided tests were run. Using regular LLVM all tests passed without any issue. In contrast, using Minotaur at least two tests, namely t-root and t-perfpow, failed. This indicates correctness issues in Alive2 or in how Minotaur uses Alive2.

We decided to ignore the failing tests and still proceed to benchmark libgmp. An improvement in runtime, by using Minotaur, could not be shown in the benchmark (1). There could be multiple reasons for this. One could be that any of the previous issues, which we were not able to resolve, caused this. Alternatively, most of the relevant optimization found by Minotaur in LLVM 18 are now already present in LLVM 20. This is not entirely impossible, since the authors were able to get some of the discovered optimizations merged into LLVM. Furthermore, there were likely more improvements done between LLVM 18 to 20, making it even more difficult to discover optimizations. The high variance between the single benchmarks could either stem from Minotaur's rewrites or be caused by the system executing the benchmark. Nevertheless, the overall results did not change significantly between different benchmark runs. Another interesting observation is that about half of the benchmark programs were slower

[1] Link to Minotuar Version
[2] Link to Alive2 LLVM Interpreter Section
[3] Link to libYUV Version

and half faster than the LLVM baseline. This seems to indicate an issue with the cost model used to estimate the found substitutions.

### 3.3 libYUV

In order to be able to even build libYUV and to not run into constant crashes in Alive2's LLVM execution tool during the compilation phase, its third party dependencies were compiled without Minotaur. This would probably not have influenced the results too much since those dependencies seemed to be used mainly for auxiliary functionality. Due to the third party dependencies not being compiled by Minotaur, which the authors seemingly did, comparing compile times makes little sense. We were not able to run the benchmark suite of libYUV, which doubles as its test suite, since it was failing the tests when using Minotaur. Similar to the results for libgmp, the regular LLVM build passed the tests with no issues.

### 3.4 Small LLVM IR Samples

These results from libgmp and libYUV prompted us to investigate further. We decided to try to reproduce some of the optimizations for short LLVM IR code snippets listed in section 5.6 from the original paper as examples. Since the original C source code for these is not provided, we attempted to optimize the IR directly. However, we observed that running optimizations on LLVM IR seemingly produces different results than compiling the original C. This mismatch could potentially explain the poor results obtained on the snippets.

In 5 of the 9 snippets extracted from the paper, Minotaur failed to produce any optimization. Only in the case of example 3 Minotaur successfully generated optimized code. The presented optimization differed from the one in the paper but such difference could be explained with the use of a different CPU. In the case of example 6, LLVM with the `-O3` flag was able to produce the expected optimization but Minotaur failed to do so. Finally, in examples 7 and 8 Minotaur produced new code which was more costly than the original, non-optimized code according to LLVM MCA. In example 7, however, LLVM with the `-O3` flag produced an optimization which was different from the one in the paper but equally as efficient.

We also attempted to reproduce optimizations reported by the paper's authors in LLVM's GitHub issues. In most cases tested [4] [5] [6] [7] [8], both Minotaur and LLVM were able to produce the expected optimizations. However, in one case [9] Minotaur failed to do so. Only in one of the tested issues [10] Minotaur produced an optimization that LLVM could not by eliminating an entire function.

[4] Issue 88168
[5] Issue 86079
[6] Issue 86068
[7] Issue 86063
[8] Issue 86061
[9] Issue 88030
[10] Issue 118114

### 3.5 Conclusion

In conclusion, it can be said that we were not able to reproduce the results of the paper. The builds took significantly longer than expected. We also found issues with the correctness of the applied optimizations. Our runtime benchmark of libgmp did not show an improvement to the average runtime. To reproduce the results from the paper, one would have to fix the caching, the correctness issues and likely also adjust the parameters to allow for a longer more thorough search for optimizations.

## 4 Research Ideas

### 4.1 Improved caching

As previously discussed, caching, which should be a key mechanism for reducing compile time, does not appear to operate very effectively in the current version of Minotaur. A potential reason is that cached rewrites are keyed on the literal structure of extracted cuts, causing near misses on semantically equivalent cuts. A possible approach to this is the introduction of a canonicalization layer, so that identical computations map to the same key. This could be implemented through the normalization of SSA identifiers, sorting of operands in commutative instructions and the standardization of other elements like vector masks. The impact of these changes could be evaluated by measuring improvements in cache hit rates, reduction of solver timeouts and compilation times, as well as by checking whether more rewrites are successfully applied across different programs.

### 4.2 Dynamic Cutting Depth using Profile Guided Optimization (PGO)

At the moment, one major down side of Minotaur are its long compile times. Even for small projects, which usually compile within seconds, it can take thousands of times longer, making it hours or even days. Improving on compile times could enable a more practical use of Minotaur. The main challenge is how to save on time while compiling and not to lose the benefits of Minotaur, i.e. runtime performance improvements. To achieve this, one could try, as the paper suggested, to dynamically set the cutting depth. Determining the cutting depth for each cut is hard since one would have to know how critical a piece of code is to the overall runtime performance. One simple way of doing this would be to first build a program without Minotaur and then benchmark it while gathering data with a profiler. This profiling data would then be used in a second compilation of the program, this time also using Minotaur, to determine the cutting depth dynamically.

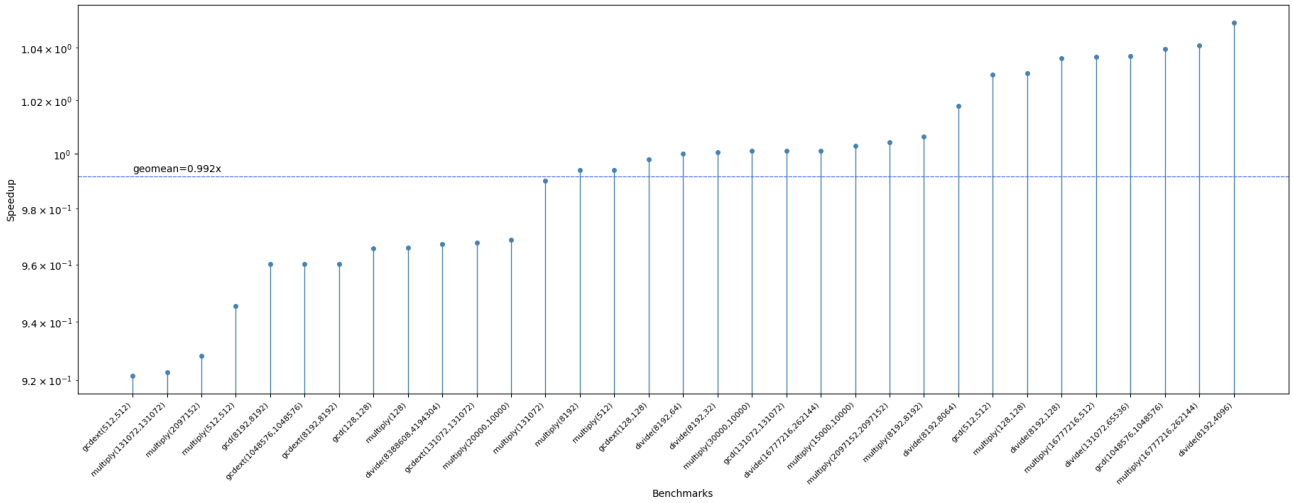


Figure 1: Our libgmp benchmark results

In theory, two benefits could be derived from this approach, namely reducing total compile time, by not spending time on unimportant code, e.g. code that is only ever executed once, and secondly, it could also enable Minotaur to spend more time on important code impacting performance. Changes in compile time between the paper's version of Minotaur and the augmented version would be the first important metric. This is not enough, one would also have to measure the runtime of the produced code in order to determine if this idea delivers any benefits. Ideally, the suggestion would improve compile times while keeping the runtime of a program constant or potentially improving it.

## 4.3 Arm + NEON Support

Currently, Minotaur only supports x86 architectures. While it contains a large amount of x86, intrinsics it still uses LLVM IR which is platform-agnostic. When considering ARM, LLVM support for NEON is more robust and more similar to x86 used in Minotaur than support for SVE. Adding ARM support would entail implementing ca. 100 intrinsics (the x86 version of Minotaur uses 165), of which many are type duplicates, i.e. 32 bit and 64 bit variants. While this seems feasible, it would also require modifying the register allocator and verifying the formal verification in Alive2 to support ARM specific instructions. LLVM MCA supports ARM already, so a similar cost model could be applied. As a result, this seems to be a significant engineering effort. Benchmarks similar to the ones used in the paper could be used to evaluate the performance of Minotaur on ARM hardware. Adding ARM support would increase the applicability of Minotaur and allow it to work on more hardware platforms.

## 4.4 Improved Introspection

Currently, Minotaur is largely a black box. While it is possible to inspect the transformations via `cache-dump` of the redis database, the resulting output is difficult to read. An improvement would be better introspection tools to allow the user to find and inspect rewrites easily. The cache can be made queryable and interactive via a graphical interface. This goes hand in hand with statistics gathering about which cuts produce the most optimizations, which optimizations are most common and which optimizations produce the highest speedup in the benchmarks. Instead of raw LLVM IR, one could display the rewrites using e.g. Graphviz to visualize the cuts and their optimizations. Furthermore, a snippet view could show code before and after optimizations. To evaluate, we would do an internal user study in our group and gather feedback. This would make Minotaur and its optimizations more accessible to researchers and make it less cumbersome to use.

## References

[1] Zhengyang Liu, Stefan Mada, and John Regehr. Minotaur: A SIMD-oriented synthesizing superoptimizer. *Proceedings of the ACM on Programming Languages (OOPSLA2)*, 8(OOPSLA2):1–25, October 2024. doi: 10.1145/3689766. https://doi.org/10.1145/3689766.

[2] Raimondas Sasnauskas, Yang Chen, Peter Collingbourne, Jeroen Ketema, Jubi Taneja, and John Regehr. Souper: A synthesizing superoptimizer. *CoRR*, abs/1711.04422, 2017. URL http://arxiv.org/abs/1711.04422.

[3] Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. *CoRR*, abs/1211.0557, 2012. URL http://arxiv.org/abs/1211.0557.